

TECHNICAL PLAN / 2026-09-22

# DiscoveryBench 与 CORE-Bench 适配方案

面向 RAC × AI Scientist 的任务接入、执行隔离与独立评测。覆盖 ARK、Agent Laboratory、EvoScientist 三个现用宿主，以及 N0-R3 四个实验条件。

## 本文件的状态

这是根据仓库阅读、合作者运行总结和公开实现整理的实施设计。本文未执行 benchmark 下载、代码适配、付费实验或部署；任务清单、具体版本和预算将在实施时冻结。

## 目标与首版范围

| 维度     | DiscoveryBench            | CORE-Bench                     |
|--------|---------------------------|--------------------------------|
| 主要能力   | 数据分析、假设形成与发现              | 科学代码的计算复现                      |
| 首轮开发范围 | real/train 中约 10 个 CPU 任务 | 约 5 个 Python、无 GPU 的开发 capsule |
| 最终交付   | discovery_result.json     | report.json                    |
| 评分来源   | 原作者评分函数的固定版本              | 固定版本 HAL Hard 评分实现             |
| 正式评估   | 冻结方案后使用独立 test 任务         | 使用与开发阶段分离的 capsule             |

## 整体路线

公共接口改造 → DiscoveryBench 闭环 → 隔离任务环境 → CORE-Bench 闭环 → N0-R3 对照验证。先用每个 benchmark 的一个任务验证，再扩大任务数和 seeds。

## 三项不变的实验约束

- N0 继续调用完整宿主原生生命周期；R1-R3 继续使用共享 RAC runner。
- 同一 host/task/model/seed 的四个条件共享公开输入、交付要求、工具权限与生命周期预算。
- 隐藏答案和官方 scorer 留在独立评分环境；R3 verification 仍是 advisory，不触发自动重试或回滚。

阅读导航：第 2 页公共接口；第 3 页 DiscoveryBench；第 4 页 CORE-Bench；第 5 页宿主与公平性；第 6 页实施与验收；第 7 页版本、待定项和来源。

IMPLEMENTATION PROPOSAL / 02

公共接口与代码改造

目标：将 benchmark 差异封装为任务数据和评分接口，维持共享协调策略。

| 对象               | 职责与关键内容                                                                                    |
|------------------|--------------------------------------------------------------------------------------------|
| TaskSpec         | benchmark_id、task_id、split、公开文件清单、output_spec、runtime_profile、资源与网络要求；不得携带 gold/reference。 |
| BenchmarkAdapter | 读取任务、构造公开工作区、检查输入边界、声明提交格式、读取提交、调用评分器。                                                     |
| ScoreResult      | 原生指标、评分状态、错误信息、评分器版本、输入/提交哈希和外部 judge 用量。                                                  |

```
load_task(...)
materialize_public_workspace(...)
validate_public_workspace(...)
required_outputs(...)
read_submission(...)
score_submission(...)
```

| 现有模块                      | 计划改动                                                                               |
|---------------------------|------------------------------------------------------------------------------------|
| benchmark.py              | 封装现有 ResearchClawBench 逻辑；新增 benchmarks 包与注册表，分别实现三个 adapter。                      |
| cli.py                    | prepare-task 增加 benchmark 选择；run-one 读取 TaskSpec；score-episode 从 episode 元数据分派评分器。 |
| matrix.py / config.py     | 去掉固定 tasks/<id> 路径假设；配置、实验 ID 和计划哈希纳入 benchmark、split 与完整任务 ID。                    |
| schemas.py / artifacts.py | 增加 OutputSpec 与结构化提交记录；从声明的文件识别最终提交，记录 artifact hashes。                            |
| policy.py                 | 按任务声明构造证据要求；支持 JSON 提交，避免把所有最终产物都判定为 Markdown 论文。                                  |
| hosts/*                   | 从统一任务对象取得提示、路径和输出要求；保持宿主调用、状态翻译和用量统计职责。                                            |

区分两种 contract

OutputSpec 是所有 N0-R3 条件共同的最终交付要求；WorkContract 是仅 R3 使用的逐步工作范围。最终 JSON 只在对应交付阶段核查，不要求每一跳都产出最终答案。

兼容策略：旧命令默认保持 ResearchClawBench 语义。新任务不再依赖 task\_info.json 的专有格式；episode 元数据保存 benchmark identity，评分时避免人工选错 adapter。

## IMPLEMENTATION PROPOSAL / 03

# DiscoveryBench：输入、提交与评分

首版采用 allenai/discoverybench 原作者实现。真实数据的原始 split 为 train/test。[1][2]

## 01 公开任务包

从原始 agent 可见输入中按白名单抽取 discovery goal、数据文件和允许的 metadata/列说明。完整 metadata 不直接复制；gold hypothesis、gold workflow 与评分专用材料只留在私有侧。同一数据集的不同问题必须拥有不同 task\_id。

沿用现有宿主的 code/、outputs/、report/ 布局。开发集优先选择数据可完整本地提供、无 GPU、无登录依赖且参考流程可在约定资源内运行的任务；覆盖多个领域，并记录纳入/排除原因。

## 02 提交格式

```
{
  "hypothesis": "...",
  "workflow": "...",
  "evidence": [
    {"claim": "...", "artifact": "outputs/result.csv"}
  ]
}
```

固定读取 discovery\_result.json。hypothesis 与 workflow 传入官方评分器；evidence 用于核查与审计，不计入官方分数。宿主在原有生命周期内生成 JSON；只做确定性的格式读取/转换，不追加模型补写。

## 03 独立评分

封装官方评分函数，保留 final\_score 以及上下文、变量和关系匹配等原生结果。workflow 是评分输入，不自行声明存在独立的官方 workflow\_score。[3]

官方旧入口固定了旧模型名称。provider 适配需覆盖全部评分调用，保留提示与算法，并固定 evaluator model、temperature、并发和重试。更换 judge 时记录版本，不直接视为原论文同口径分数。[4]

## 04 最小验收

- 参考提交、明显错误提交、空提交与缺失字段均得到可解释结果；参考提交只存在于评分环境。
- 模拟 provider/解析错误不会静默变为数值零分；原始响应与评分失败原因可追踪。
- 一个开发任务完成“准备 → 宿主 N0 → 提交读取 → 独立评分 → 归档”闭环后，才扩展到其他条件。

IMPLEMENTATION PROPOSAL / 04

# CORE-Bench：保持 Hard 的评测含义

首版聚焦 Python、无 GPU 的 Hard capsule；读取原生问题并生成 report.json。[5][6]

## 01 输入过滤与目录映射

预下载 capsule 并记录哈希；每次 episode 使用全新副本。按选定 HAL Hard 版本生成文件清单，执行其原生过滤规则，排除该版本规定的预计算结果、环境定义、复现辅助文件等。保持 capsule 内部相对路径，明确记录任务根目录映射。[7]

## 02 双环境设计

| 环境           | 配置边界                                              |
|--------------|---------------------------------------------------|
| 宿主环境         | 预装 AI Scientist 自身所需依赖，运行模型调用与原生工作流。              |
| capsule 执行环境 | 从固定 Hard 基础环境开始。任务依赖由 agent 自行识别和安装；不得继承宿主的科学计算包。 |
| 私有参考/评分环境    | 运行参考复现和 grader；其结果、gold 及已解决的依赖环境不得流入正式任务工作区。     |

新增通用 TaskRuntime：命令执行、文件读写、退出码、超时与日志。三个宿主的任务执行工具转发到独立执行环境；由外层控制程序管理容器，宿主不挂载 Docker socket。需要核查全部执行路径，避免部分代码仍在宿主 Python 环境中运行。

基础镜像和包源策略可提前固定；不能预装通过参考答案求出的整套任务依赖。若使用依赖镜像源，所有条件共享相同策略，安装动作和结果可追踪。Hard 要求 agent 恢复任务环境，这部分必须保留。[6]

## 03 原生提交与评分

```
{
  "Exact question from the task": 0.123,
  "Another exact task question": "answer"
}
```

在任务根输出 report.json，键保持原始问题，值为答案。保留执行日志、安装记录和结果文件；report.md 仅作辅助说明。封装固定 HAL 版本的数值、文本和列表比较规则，保留逐题结果与“整个任务全部答对”的成功率。[5][7]

Hard 验收门槛

先用私有参考运行验证任务可复现，再核对公开文件清单、环境隔离和 grader 一致性。若某宿主仍能直接使用其预装任务依赖，该组合尚不能按本方案宣称完成 Hard 适配。

IMPLEMENTATION PROPOSAL / 05

宿主适配与实验可比性

宿主翻译任务与产物；公共策略控制协调；官方评分在 episode 结束后独立执行。

| 宿主               | 具体接入位置与约束                                                                          |
|------------------|------------------------------------------------------------------------------------|
| ARK              | 把任务说明和 JSON 交付要求送入研究、实验、写作与审阅上下文；保留原生顶层 lifecycle 和角色调用。                           |
| Agent Laboratory | 通过 notes/阶段提示传递 OutputSpec；ResearchClawBench 专用数据与文献补丁仅在其对应任务启用；任务代码走 TaskRuntime。 |
| EvoScientist     | 将固定围绕 report/report.md 的交付检查改为读取任务输出规格；保留原生 Deep Agent 生命周期和已有内部修订语义。              |

交付检查与终止状态分开记录

保留 native\_status、submission\_status 和 score\_status。原生流程返回不等于提交有效，提交有效也不等于评分已完成。新 benchmark 的有效 JSON 提交不因缺少论文式长报告而被禁止评分；报告只作额外审计产物。

R3 在分析阶段核查代码/结果，在最终交付阶段核查声明的 JSON。验证结论继续进入 ledger 与后续上下文；不读取隐藏答案，不因校验失败追加重试、回滚或模型格式修复。

预算制定

- 先在开发任务上测量 N0 的 wall time、tokens 和 calls，再预先冻结同一 host/task/model 的 N0-R3 共同 ceiling 与 headroom。不得根据条件结果事后调参。
- 当前 N0 不进入 RAC loop，RAC hop 数通常记为 0；不能用该数与 R1-R3 直接等预算对齐。max\_hops 是额外执行上限，成本比较仍使用实际用量。
- 每个条件统一模型、公开输入、权限、依赖访问和资源限制；episode 内协调用量计入共同预算。外部 benchmark judge 成本单列。

结果与错误语义

| 情况           | 记录方式                                                      |
|--------------|-----------------------------------------------------------|
| 任务失败、超时或预算耗尽 | 保留 episode、已有产物、实际 usage 和终止原因；若存在提交则仍尝试评分。               |
| 无效或缺失提交      | 记录 submission_status；官方规则若明确判失败，保留该原生判定，不能从实验分母剔除。        |
| 评分服务或解析故障    | score_status 标记未完成，分数为 null 并附 error；保留原始输出，避免用 0 隐藏评分故障。 |

汇总同时报告原生分数、评分覆盖率、任务成功率、成本与时长。不得只报告有完整分数的成功样本均值；未完成评分应单列并补评。

IMPLEMENTATION PROPOSAL / 06

实施顺序、验证规模与验收

以下均为计划中的交付物。付费验证在授权实施后进行，当前不启动。

| 阶段               | 主要交付                               | 通过条件                           |
|------------------|------------------------------------|--------------------------------|
| A 公共接口           | TaskSpec、adapter 注册、CLI、矩阵和 RCB 兼容 | 现有离线测试通过；RCB 任务与评分回归路径完成核查。    |
| B DiscoveryBench | 公开任务包、提交校验、独立 scorer               | 一个开发任务完整闭环；输入不泄露答案；评分错误不伪装为零分。 |
| C 执行环境           | TaskRuntime 和三个宿主工具接入              | 任务代码与宿主依赖隔离；退出码、超时、安装与文件产物可追踪。 |
| D CORE-Bench     | Hard 文件过滤、capsule 与固定 grader       | 参考复现成功；相同提交的适配结果与固定原评分器一致。     |
| E 对照验证           | 两 benchmark、三宿主、N0-R3              | 相同公开输入与预算；条件机制、产物和评分记录全部可审计。   |

从单 cell 到完整组合

先运行 ARK × 一个开发任务 × N0，再完成 Agent Laboratory、EvoScientist 的 N0；随后扩展到 R1-R3。对两个 benchmark 分别执行这一顺序。R1-R3 每个正式 cell 使用新 SharedNet Room；N0 不读取 Room 配置。

| 层级         | 规模                                                                   | 用途           |
|------------|----------------------------------------------------------------------|--------------|
| 接口验证       | 2 benchmarks × 1 task × 3 hosts × 4 conditions = 24 cells            | 确认适配闭环与协调语义  |
| 开发 / pilot | (10 Discovery + 5 CORE) × 3 hosts × 4 conditions = 180 episodes/seed | 观察资源与故障类型    |
| 正式实验       | 任务清单、seeds 和预算另行冻结                                                   | 使用与开发阶段分离的任务 |

有针对性的回归测试

- 输入边界：gold/答案字段不进入公开目录；路径穿越、越界链接和危险归档成员被拒绝。
- 评分适配：同一 fixture 与固定 scorer 一致；缺失、错误类型和 provider 故障的状态区分正确。
- 运行语义：N0 绕过 RAC；R1 固定 successor；R2 允许路由；R3 advisory 不改变重试/回滚行为。
- 可复现性：全新工作区与隔离环境、完整 usage/哈希、失败产物归档；RCB 旧路径不回退。

环境 build 与 reference smoke 不替代真实宿主验证。中途改过代码的试跑不并入冻结版本的正式结果；适配验收不要求 agent 取得高分。

IMPLEMENTATION PROPOSAL / 07

待冻结事项与参考来源

本方案基于当前仓库与提供的运行总结；公开实现核对日期为 2026-09-22。

| 待冻结项      | 确定方法                                                                                   |
|-----------|----------------------------------------------------------------------------------------|
| 版本与可复现性   | 锁定 benchmark、dataset、scorer、宿主与 RAC revision；记录镜像 digest、输入和提交哈希；本地兼容修复单独记录。           |
| 任务清单与硬件   | 按 CPU/GPU、语言、数据可用性、内存和参考时长制定规则；在查看条件成绩前冻结 ID、纳入/排除原因。                                  |
| 预算与 judge | 开发 N0 测量后确定共同预算；单独冻结 DiscoveryBench judge 配置与错误/重试规则。                                  |
| 现有流程前置核查  | 运行总结指出 ResearchClawBench judge 仅直连 smoke 已确认；完整 episode → scorer → score.json 需补齐回归验证。 |

DiscoveryBench 首版采用原作者数据与 scorer，不混用 AstaBench 的 split 命名或评分。CORE-Bench 采用固定 HAL Hard 快照；HAL 已归档，需锁定具体版本。[2][7][8]

公开资料

[1] [DiscoveryBench 项目与输入/输出入口](#)  
<https://github.com/allenai/discoverybench>

[2] [DiscoveryBench real 数据的 train/test 目录](#)  
<https://github.com/allenai/discoverybench/tree/main/discoverybench/real>

[3] [DiscoveryBench 官方评分实现](#)  
[https://github.com/allenai/discoverybench/blob/main/eval/new\\_eval.py](https://github.com/allenai/discoverybench/blob/main/eval/new_eval.py)

[4] [DiscoveryBench 评分 CLI 与模型配置](#)  
[https://github.com/allenai/discoverybench/blob/main/discovery\\_eval.py](https://github.com/allenai/discoverybench/blob/main/discovery_eval.py)

[5] [CORE-Bench 原作者运行与 report.json 协议](#)  
<https://github.com/siegelz/core-bench/blob/main/README.md>

[6] [HAL 对 CORE-Bench Hard 的定义](#)  
[https://hal.cs.princeton.edu/corebench\\_hard](https://hal.cs.princeton.edu/corebench_hard)

[7] [HAL 的 Hard 输入过滤与评分实现](#)  
<https://github.com/princeton-pli/hal-harness/blob/main/hal/benchmarks/corebench.py>

[8] [HAL 仓库归档状态](#)  
<https://github.com/princeton-pli/hal-harness>

项目资料

本地仓库：RAC-AI\_Scientist；重点参考 README、architecture、CLI、policy、runner、三个 active host 适配器与测试。

合作者总结：validated-runbook-and-future-benchmarks.zh-CN.md，更新于 2026-09-21。服务器侧状态按该总结引用，未在本文档制作中重新验证。